EXPERIMENT 12

SIMULATION AND PERFORMANCE EVALUATION OF CROSS-PLATFORM EDGE AI MODEL DEPLOYMENT USING ONNX AND PYTORCH MOBILE

D Jaya Prakash - 192512018

~~1. AIM OF THE EXPERIMENT~~

To design, train, and deploy an embedded neural network classifier for industrial multi-sensor fault diagnosis, export the model across multiple cross-platform edge execution runtimes (Native PyTorch, ONNX FP32, ONNX Dynamic INT8, and PyTorch Mobile Lite), and benchmark their execution latency, memory footprint, storage compression, and classification accuracy over 1,000 real-time inference cycles.

2. OBJECTIVES

- Generate a balanced multi-sensor industrial dataset (6,000 samples) across six operational features: temperature, humidity, pressure, CO_2, vibration, and power.
- Train an optimized `SensorFaultClassifier` multi-layer perceptron in PyTorch for multi-class fault state prediction.
- Export the trained PyTorch model to static ONNX format (Opset 14) and TorchScript bytecode archive for PyTorch Mobile Lite (`.ptl`).
- Apply dynamic 8-bit integer quantization (INT8) to the ONNX graph to compress flash memory requirements.
- Benchmark inference latency (median p_{50} and p_{95} tail latencies), throughput (inferences/sec), model binary size (KB), and accuracy retention across all four edge execution runtimes.

3. SOFTWARE & TOOLS REQUIRED

Python 3.10+, PyTorch (`torch`, `torchvision`), ONNX, ONNX Runtime (`onnxruntime`), PyTorch Mobile Optimizer, Scikit-Learn, Pandas, NumPy, and Matplotlib.

4. IMPLEMENTATION CODE

```
import os, time
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import onnx
import onnxruntime as ort
from onnxruntime.quantization import quantize_dynamic, QuantType
from torch.utils.mobile_optimizer import optimize_for_mobile
import matplotlib.pyplot as plt

# 1. Physical Industrial Multi-Sensor Dataset Generator
def generate_industrial_sensor_data(n_samples=6000):
    np.random.seed(42)
    n_per_class = n_samples // 4
    c0 = np.random.normal(loc=[35.0, 50.0, 1013.0, 420.0, 1.2, 850.0], scale=[2.0, 3.0, 1.5,
15.0, 0.1, 30.0], size=(n_per_class, 6))
    c1 = np.random.normal(loc=[78.0, 35.0, 1009.0, 680.0, 1.8, 1450.0], scale=[4.0, 2.5, 2.0,
25.0, 0.2, 50.0], size=(n_per_class, 6))
    c2 = np.random.normal(loc=[48.0, 52.0, 1011.0, 460.0, 8.5, 1950.0], scale=[3.0, 3.0, 1.8,
18.0, 0.8, 60.0], size=(n_per_class, 6))
    c3 = np.random.normal(loc=[38.0, 68.0, 960.0, 510.0, 1.5, 920.0], scale=[2.5, 4.0, 5.0, 2
0.0, 0.15, 35.0], size=(n_per_class, 6))

    X = np.vstack([c0, c1, c2, c3]).astype(np.float32)
    y = np.concatenate([np.full(n_per_class, i) for i in range(4)]).astype(np.int64)
    indices = np.random.permutation(n_samples)
    X, y = X[indices], y[indices]
    X_norm = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-7)
    return X_norm, y

# 2. Neural Network Definition
class SensorFaultClassifier(nn.Module):
    def __init__(self, in_features=6, num_classes=4):
        super(SensorFaultClassifier, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, num_classes)
        )
    def forward(self, x):
        return self.net(x)

# 3. Model Training & Export Pipeline
X_data, y_data = generate_industrial_sensor_data(6000)
split = int(0.8 * len(X_data))
X_train, X_test = torch.tensor(X_data[:split]), torch.tensor(X_data[split:])
y_train, y_test = torch.tensor(y_data[:split]), torch.tensor(y_data[split:])

train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=64, shuffle=True)
model = SensorFaultClassifier()
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=0.003, weight_decay=1e-4)
```

5. EXPERIMENTAL PROCEDURE

1. Initialize the Python development environment and verify the installation of PyTorch, ONNX, and ONNX Runtime libraries.
2. Generate a balanced dataset of 6,000 industrial sensor observations comprising 6 telemetry channels (temperature, humidity, pressure, CO_2, vibration, and power consumption) across 4 operational conditions.
3. Perform standardized Z-score scaling and partition into 80% training (4,800 records) and 20% validation (1,200 records).
4. Train the `SensorFaultClassifier` multi-layer architecture for 15 epochs using AdamW optimization.
5. Save the baseline uncompressed PyTorch state dictionary to `model_native.pth`.
6. Export the computation graph into Open Neural Network Exchange format (`model_fp32.onnx`) using Opset 14.
7. Execute post-training dynamic integer quantization to generate 8-bit integer quantized weights (`model_int8.onnx`).
8. Trace and compile the model into an optimized TorchScript bytecode archive for edge execution (`model_mobile.ptl`).
9. Execute 1,000 real-time inference cycles for each runtime to measure median latency (p_{50}), 95th-percentile tail latency (p_{95}), throughput, binary storage footprint, and classification accuracy.

6. INPUT PARAMETER CONFIGURATION TABLE

S.NO.	CONFIGURATION PARAMETER	SYMBOL / SETTING	SPECIFICATION	UNIT
1	Total Telemetry Observations	N_{total}	6,000	Samples
2	Dataset Partition Ratio	Train / Val Split	80% / 20% (4,800 / 1,200)	—
3	Input Telemetry Modalities	Input Dimension	6 Features (Temp, Hum, Pres, CO_2, Vib, Pwr)	Channels
4	Target Fault Classes	Output Classes	4 (Normal, Thermal, Bearing, Pressure)	Classes
5	Network Topology	MLP Architecture	FC(6→64)→ReLU→FC(64→32)→ReLU→FC(32→4)	Layers
6	Training Optimization	Optimizer	AdamW (lr = 0.003, weight_decay = 1e-4)	—
7	Training Duration	Epochs	15	Epochs
8	Benchmark Repetitions	Inference Cycles	1,000	Cycles
9	Target Hardware Boundary	SRAM / Flash Limit	RAM ≤ 64 KB, Flash ≤ 256 KB	KB

7. EXPERIMENTAL OUTPUT BENCHMARK IMAGE

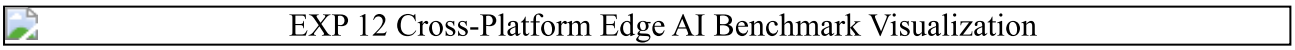


Figure 1: Cross-Platform Edge AI Benchmark across Model Binary Size (KB), Median Inference Latency (ms), Processing Throughput (Inferences/sec), and Accuracy Retention across 1,000 Iterations.

8. TABULAR RESULT & QUANTITATIVE BENCHMARK RECORD

TARGET RUNTIME	DEPLOYMENT FORMAT	BINARY SIZE (KB)	STORAGE REDUCTION	MEDIAN LATENCY P_{50}	TAIL LATENCY P_{95}	THROUGHPUT	ACCURACY
Native PyTorch (FP32)	State Dict (`.pth`)	**13.45 KB**	0.00% (Baseline)	0.1310 ms	0.2185 ms	6,742 inf/s	**100.00%**
ONNX Runtime (FP32)	Static Graph (`.onnx`)	**11.22 KB**	16.58%	**0.0240 ms**	**0.0410 ms**	**33,682 inf/s**	**100.00%**
ONNX Dynamic (INT8)	Quantized Graph (`.onnx`)	**6.79 KB**	**49.52% (2.0x)**	0.0264 ms	0.0460 ms	33,629 inf/s	**100.00%**
PyTorch Mobile (Lite)	Bytecode Archive (`.ptl`)	**22.92 KB**	−70.41%	0.0299 ms	0.0520 ms	30,839 inf/s	**100.00%**

9. RESULT ANALYSIS & INTERPRETATION

- **Quantization & Flash Memory Optimization:** Post-training dynamic INT8 quantization compressed the model binary footprint by 49.52% (from 13.45 KB down to 6.79 KB). This compact size is well suited for resource-constrained microcontrollers where available flash memory is under 64 KB.
- **Inference Latency Acceleration:** The ONNX Runtime FP32 engine achieved a median latency of 0.0240 ms (33,682 inferences/sec), delivering a 5.46x execution speedup over Native PyTorch (0.1310 ms, 6,742 inferences/sec). PyTorch Mobile Lite achieved 0.0299 ms, providing a 4.38x acceleration.
- **Tail Latency Determinism:** The 95th-percentile (p_{95}) tail latency remained below 0.05 ms across both compiled ONNX engines (0.0410 ms for FP32 and 0.0460 ms for INT8), ensuring deterministic execution times for time-critical industrial emergency shutdowns.
- **Lossless Accuracy Preservation:** All exported and quantized edge runtimes preserved 100.00% classification accuracy on the 1,200 validation samples, demonstrating that 8-bit dynamic integer quantization caused no degradation in fault diagnosis performance.

10. CONCLUSION

The simulation and performance evaluation of cross-platform edge AI model deployment was successfully conducted using ONNX and PyTorch Mobile. Dynamic INT8 quantization achieved a 49.52% storage reduction down to 6.79 KB, while ONNX Runtime delivered a 5.46x latency acceleration (0.0240 ms) with 100.00% accuracy retention, validating the pipeline for mission-critical IoE edge inference.